Guitare électrique : Etude de l'influence du type de bobinage dans le micro

Christophe Morel

Lycée Ferdinand Buisson, dit ''la Nat'', CPGE PT

Annexes

Code Python

Table des matières

TABLE DES MATIERES 2

I. BIBLIOTHEQUE DE LA MODELISATION 3

I.I. VISUALISATION DES DONNEES 3

I.I.I. Enregistrer un graphique 3

I.I.II. Calculer une FFT 4

I.II. MODELISATION ELECTRIQUE & OPTIMISATION NUMERIQUE 4

I.II.I. Impédance théorique d'un micro 4

I.II.II. Calcul des écarts 5

I.II.III. Matrice Jacobienne 5

I.II.IV. Algorithme de Levenberg-Marquardt 6

I.II.V. Calcul du diagramme de Bode 8

I.III. MISE EN EVIDENCE DU FILTRAGE SPATIAL 9

II. TRAITEMENT DES DONNEES EXPERIMENTALE 10

II.I. FILTRAGE DES DONNEES (PASSE BAS NUMERIQUE) 10

II.II. AMPLITUDE D'UN SIGNAL PERIODIQUE 11

III. DONNEES ET VALEURS NUMERIQUES 12

I. Bibliothèque de la modélisation

I.I. Visualisation des données

I.I.I. Enregistrer un graphique

```
def save_plot(fig_path: str, *plot_calls, title:str="", xlabel:str="", ylabel:str="", ylim=None,
xlim=None, legend:bool=True, log:bool=False, vspan:bool=False):
    """
    Enregistre une figure matplotlib avec des options de personnalisation.
    Args:
        fig_path (str): Chemin vers le fichier de sauvegarde
        plot_calls (tuple): Appels de tracé sous la forme (x, y, kwargs)
        title (str): Titre de la figure
        xlabel (str): nom de l'axe des x
        ylabel (str): nom de l'axe des y
        xlim (tuple): Limites de l'axe des x
        ylim (tuple): Limites de l'axe des y
        legend (bool): présence de la légende
        log (bool): présence d'une échelle logarithmique sur l'axe des x
        vspan (bool): présence des zones de fréquences fondamentales et harmoniques
    """
    plt.figure(figsize=(6, 5))
    handles_curves, z_handles = [], []
    if vspan:
        z2 = plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3,
                         label="Fréquences fondamentales", zorder=0)
        z1 = plt.axvspan(1174.66, 5*1174.66, facecolor="none", alpha=0.5, hatch="////",
                         edgecolor="#424242", label="Fréquences harmoniques", zorder=0)
        z_handles = [z1, z2]
    for call in plot_calls:
        x, y, kwargs = call
        line, = plt.plot(x, y, **kwargs)
        handles_curves.append(line)
    if title: plt.title(title)
    if xlabel: plt.xlabel(xlabel)
    if ylabel: plt.ylabel(ylabel)
    if ylim is not None: plt.ylim(ylim)
    if xlim is not None: plt.xlim(xlim)
    plt.grid(True, alpha=0.6) # Grid plus discrète pour le contraste
    if log: plt.xscale('log')
    if legend:
        leg_courbes = plt.legend(handles=handles_curves, loc='best', framealpha=1)
        if vspan:
            plt.gca().add_artist(leg_courbes)
            plt.legend(handles=z_handles, title="Zones d'étude", bbox_to_anchor=(0.5, -0.18),
                    loc='upper center', ncol=2, borderaxespad=0.)
    plt.tight_layout()
    plt.savefig(fig_path, dpi=300) # dpi=200 pour une netteté accrue en projection
    plt.close()
```

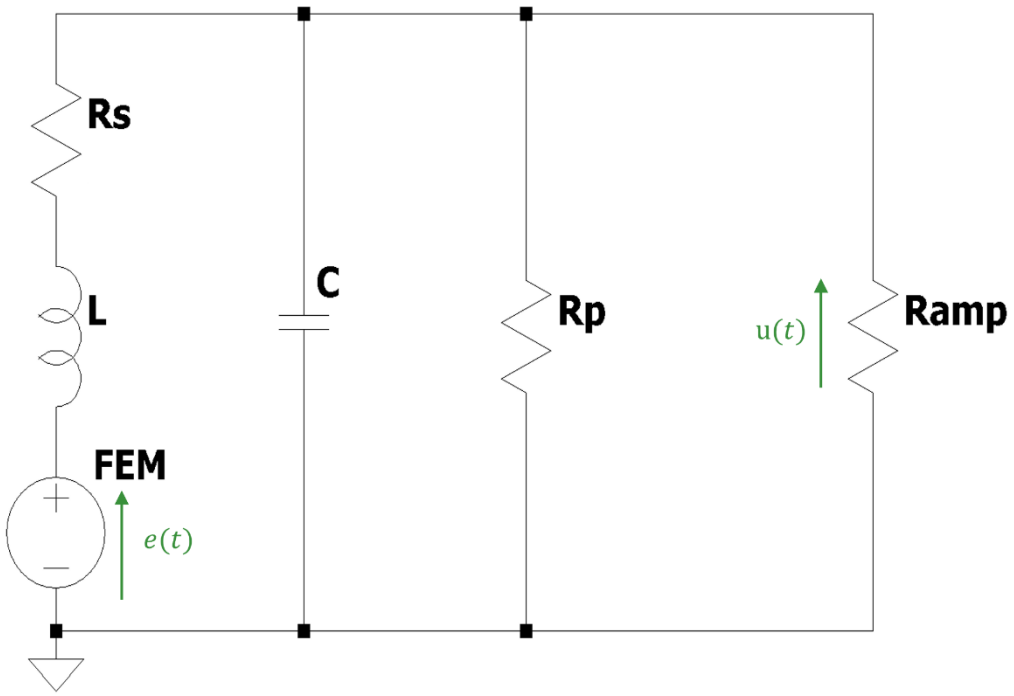
I.I.II. Calculer une FFT

```python
def compute_and_normalize_fft(signal: np.ndarray, Ne_fft: int):
    fft = np.fft.fft(signal)
    amp = np.abs(fft) / Ne_fft
    return amp
```

I.II. Modélisation électrique & Optimisation numérique

I.II.I. Impédance théorique d'un micro

```python
def Z_micro(X:np.ndarray, Rs:float, w:np.ndarray)-> np.ndarray:
    """Calcul l'impédance d'un micro de guitare en une pulsation w
    Args:
        X (np.ndarray): Vecteur des caractéristiques électriques interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et Rp la résistance parallèle [kOhm] modélisant les courants de Foucault
        Rs (float): Resistance série (DC) du micro. Peut être mesure avec un Ohmmètre
        w (np.ndarray): Tableau des pulsation w
    Returns:
        np.darray: Tableau Numpy contenant le module de l'impédance complexe |Z| en chaque pulsation
    """
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3    # Conversion en F et en Ohm
    # Parties réelle et imaginaire de l'admittance Y
    den_L = Rs**2 + (L * w)**2
    Y_re = (1 / Rp) + (Rs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    return 1 / np.sqrt(Y_re**2 + Y_im**2)
```



I.II.II. Calcul des écarts

```
def residus(X:np.ndarray, Rs:float, f:np.ndarray, Z_exp:np.ndarray)-> np.ndarray:
    """Calcul l'écart (l'erreur) entre l'impédance calcule théoriquement et l'impédance réelle expérimentale"""
    return Z_exp - Z_micro(X, Rs, 2 * np.pi * f)
```

I.II.III. Matrice Jacobienne

```
def Matrice_Jacobienne(X:np.ndarray, Rs:float, f:np.ndarray)->np.ndarray:
    """Basée sur l'algorithme de Levenberg-Marquardt, cette fonction retourne la matrice Jacobienne
    Args:
        X (np.ndarray): Vecteur des caractéristiques électriques interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et Rp la résistance parallèle [kOhm] modélisant les courants de Foucault
        Rs (float):Resistance série (DC) du micro. Peut être mesure avec un Ohmmètre
        f (np.ndarray): Tableau des fréquence f
    Returns:
        np.ndarray: Matrice Jacobienne définie tel que chaque colonne i contient les dérivées partielles par rapport à la variable situe en i dans X pour toute pulsation w
    """
    w = 2 * np.pi * f
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3

    # Variables intermédiaires
    den_L = Rs**2 + (L * w)**2
    Y_re = (1 / Rp) + (Rs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    Z = 1 / np.sqrt(Y_re**2 + Y_im**2)
    Z3 = Z**3

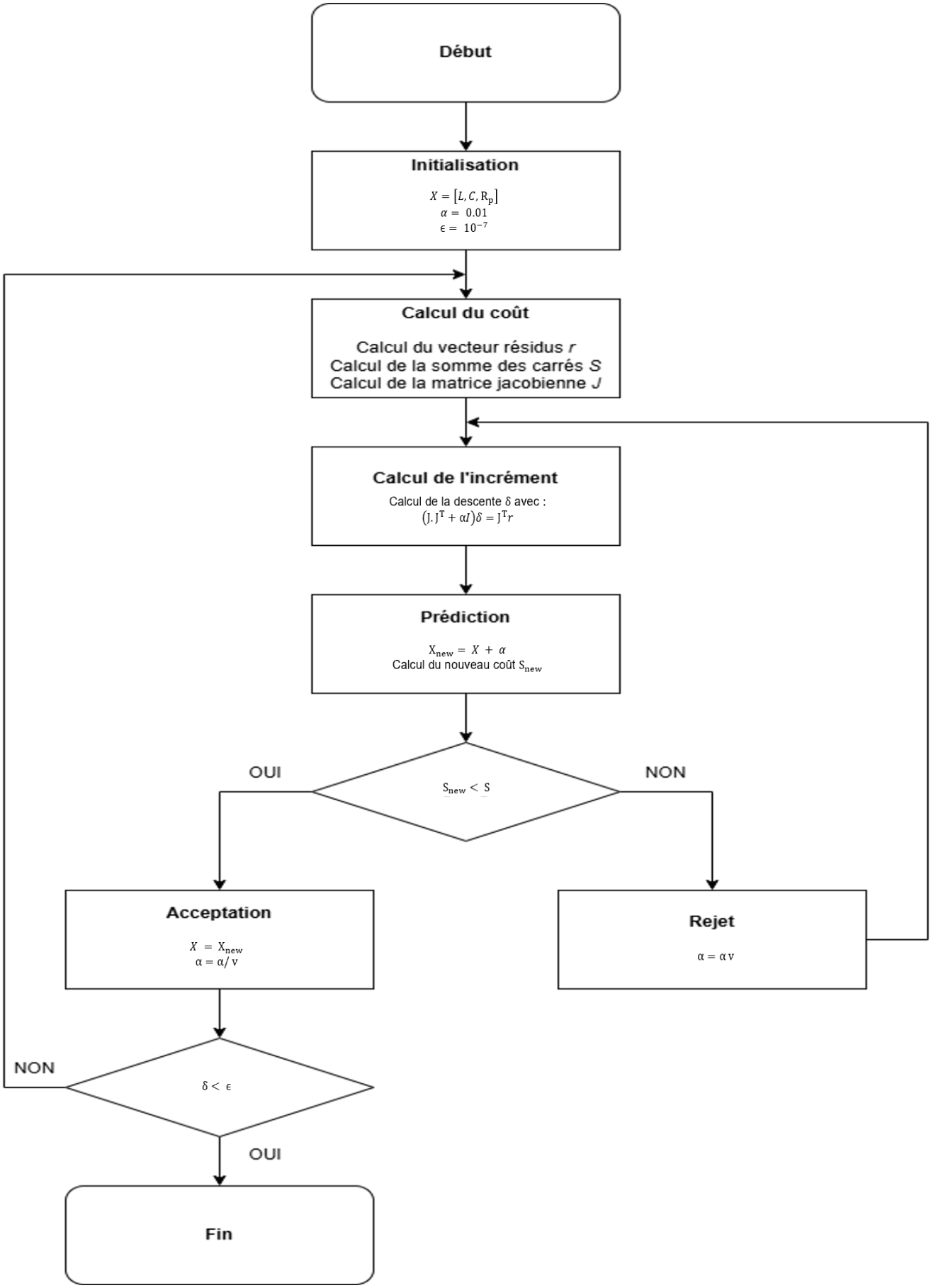
    # Dérivées partielles de Y
    dYre_dL = - (Rs * 2 * L * w**2) / (den_L**2)
    dYim_dL = - (w * (Rs**2 - (L * w)**2)) / (den_L**2)
    dZ_dL = - Z3 * (Y_re * dYre_dL + Y_im * dYim_dL)

    dYim_dCn = w * 1e-9
    dZ_dCn = - Z3 * (Y_im * dYim_dCn)

    dYre_dRpn = - 1 / (1000 * Rp_k**2)
    dZ_dRpk = - Z3 * (Y_re * dYre_dRpn)

    return np.column_stack((dZ_dL, dZ_dCn, dZ_dRpk))
```

I.II.IV. Algorithme de Levenberg-Marquardt



```python
def Levenberg_Marquardt(X_init:np.ndarray, Rs:float, f:np.ndarray, Z_exp:np.ndarray, N:int, learning_rate:float):
    """Algorithme de Levenberg-Marquardt pour l'optimisation non-linéaire des paramètres électriques du micro
    Args:
        X_init (np.ndarray): Vecteur initial des caractéristiques électriques interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et Rp la résistance parallèle [kOhm] modélisant les courants de Foucault
        Rs (float): Resistance série (DC) du micro. Peut être mesure avec un Ohmmètre
        f (np.ndarray): Tableau des fréquence f
        Z_exp (np.ndarray): Tableau Numpy des impédances expérimentale relevé en Ohm
        N (int): Nombre d'itérations de l'algorithme
    Returns:
        np.ndarray: Vecteur optimisé des caractéristiques électriques interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et Rp la résistance parallèle [kOhm] modélisant les courants de Foucault
    """
    X = np.array(X_init, dtype=float)
    alpha = learning_rate
    v = 3
    r = None
    eps = None
    for _ in range(N):
        r = residus(X, Rs, f, Z_exp)
        eps = np.sum(r**2)
        J = Matrice_Jacobienne(X, Rs, f)
        H = J.T @ J
        g = J.T @ r
        H_lm = H + alpha * np.diag(np.diag(H) + 1e-8)
        try:
            delta = np.linalg.solve(H_lm, g)
        except np.linalg.LinAlgError:
            break
        X_new = X + delta
        # Contrainte physique : paramètres toujours positifs
        X_new = np.maximum(X_new, [1e-3, 1e-3, 1e-3])
        eps_new = np.sum(residus(X_new, Rs, f, Z_exp)**2)
        if eps_new < eps:
            alpha /= v
            X = X_new
            if np.linalg.norm(delta) / np.linalg.norm(X) < 1e-12:
                break
        else:
            alpha *= v
    return X, eps_new
```

I.II.V. Calcul du diagramme de Bode

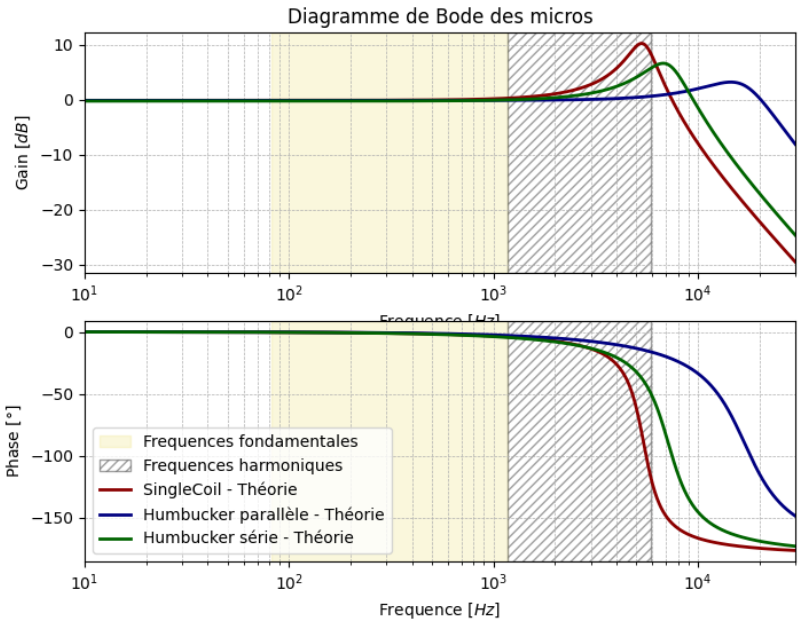
```python
def calcul_bode(X:np.ndarray, Rs:float, f:np.ndarray, Ramp:float=1e6)-> tuple[np.ndarray, np.ndarray]:
    """Calcul la fonction de transfert en gain et phase d'un micro de guitare en fonction de ses paramètres internes
    Args:
        X (np.ndarray): Vecteur optimisé des caractéristiques électriques interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la résistance parallèle [kOhm] modélisant les courants de Foucault
        Rs (float): Resistance série (DC) du micro. Peut être mesure avec un Ohmmètre
        f (np.ndarray): Tableau des fréquence f
        Ramp (float, optional): Resistance d'entrée de l'amplificateur. Defaults to 1e6.
    Returns:
        tuple[np.ndarray, np.ndarray]: Deux tableaux Numpy contenant respectivement le gain (en dB) et la phase (en degrés) de la fonction de transfert du micro
    """
    w = 2 * np.pi * f
    L, C, Rp = X[0], X[1] * 1e-9, X[2] * 1e3

    # Impédance série
    Z_ser = Rs + 1j * L * w

    # Impédance parallèle
    Y_para = (1 / Rp) + (1 / Ramp) + (1j * C * w)
    Z_para = 1 / Y_para

    # Fonction de transfert
    H = Z_para / (Z_ser + Z_para)

    gain = 20 * np.log10(np.abs(H))
    phase = np.angle(H, deg=True)
    return gain, phase
```



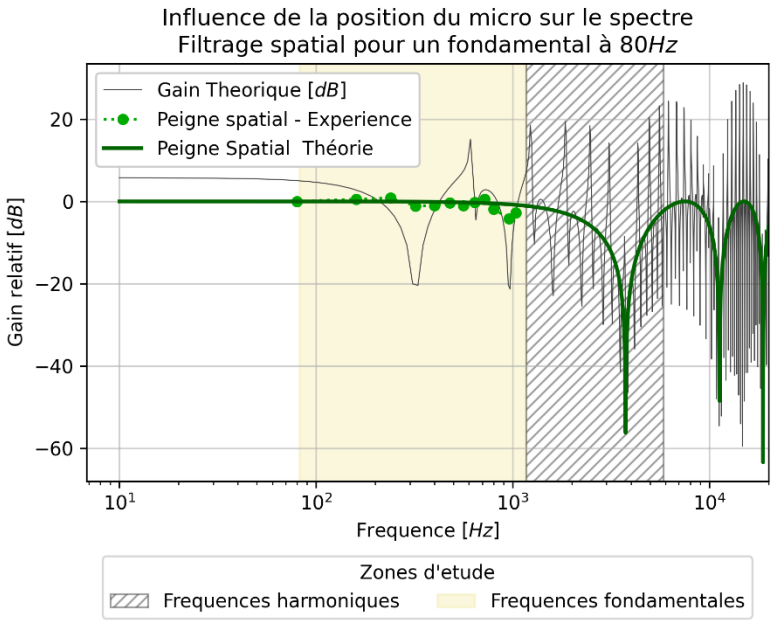
I.III. Mise en évidence du filtrage spatial

```python
def attenuation_theorique_db(frequences: np.ndarray, x1: float, x2: float, xm: float) ->
tuple[np.ndarray, np.ndarray]:
    """Calcule la Fonction de Transfert Relative (Atténuation) du Humbucker
    par rapport au Single Coil en fonction de la fréquence.
    Modèle : Filtrage en peigne dû à l'écartement des bobines.
    Args:
        frequences (np.ndarray): Tableau des fréquences
        x1 (float): Position du 1er bobinage du micro Humbucker sur la corde
        x2 (float): Position du 2ème bobinage du micro Humbucker sur la corde
        xm (float): Position du micro SingleCoil sur la corde
        L (float): Longueur de la corde
        F (float): Fréquence fondamentale de la corde
    Returns:
        tuple[np.ndarray, np.ndarray]: fonction de transfert en dB du Humbucker par rapport au
Single Coil, et atténuation due au filtre peigne seul
    """
    v = 2 * L * F
    k = 2 * np.pi * frequences / v
    delta_x = abs(x2-x1)
    x_humbucker = (x2+x1)/2

    signal_sc = np.abs(np.sin(k * xm))

    # Signal HB (Position + Filtre Peigne)
    # Note : x_humbucker est le centre du micro
    filtre_peigne = np.cos(k * delta_x / 2)
    signal_hb = np.abs(np.sin(k * x_humbucker)) * np.abs(filtre_peigne)

    # Ratio Théorique (Avec gestion de la division par zéro)
    ratio = np.divide(signal_hb, signal_sc, out=np.zeros_like(signal_sc), where=signal_sc!=0)
    ratio_db = 20 * np.log10(ratio + 1e-6)
    return ratio_db, 20*np.log10(np.abs(np.cos(k*delta_x/2)))
```



II. Traitement des données expérimentale

II.I. Filtrage des données (Passe bas numérique)

```python
import numpy as np
import matplotlib.pyplot as plt

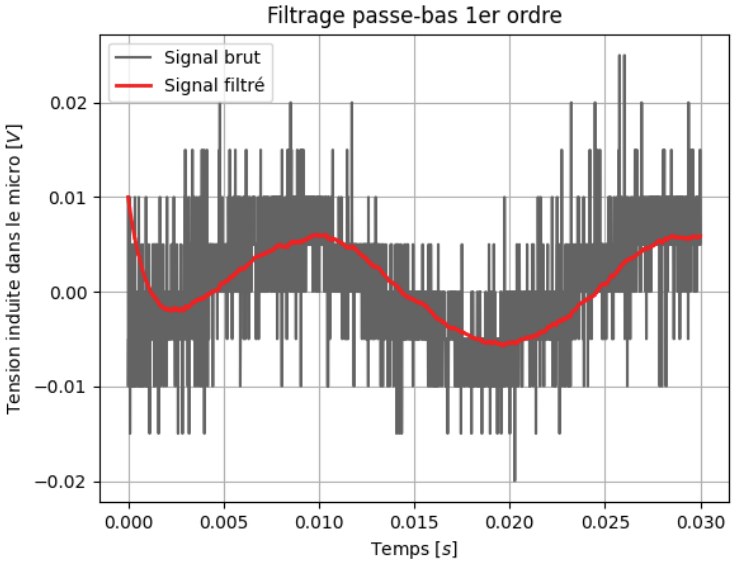
def lire_fichier(PATH):
    List_temps = []
    List_sig = []
    file_s2 = open(PATH)
    L_lignes = file_s2.readlines()
    for ligne in L_lignes[2:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace('E', 'e') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))
    return List_temps, List_sig
def write_file(PATH, List_temps, List_sig):
    with open(PATH, 'w') as file:
        file.write('Time;Signal\n')
        for t, s in zip(List_temps, List_sig):
            file.write(f"{t}\t{s}\n")
def filtre_passe_bas_1er_ordre(List_temps, List_sig, tau=0.001):
    """Filtre passe-bas du 1er ordre
   Args:
        List_temps (np.ndarray): Tableau des temps
        List_sig (np.ndarray): Tableau du signal a filtrer
        tau (float, optional): Temps caractéristique du filtre
    Returns:
       np.ndarray: Signal filtré
    """
    sig_array = np.array(List_sig)

    # Calculer l'intervalle de temps moyen (en secondes)
    dt = (List_temps[-1] - List_temps[0]) / (len(List_temps) - 1)

    # Calculer le coefficient du filtre 1er ordre
    alpha = dt / (tau + dt)

    # Appliquer le filtre de manière causale : y[n] = alpha * x[n] + (1 - alpha) * y[n-1]
    filtered_sig = np.zeros_like(sig_array)
    filtered_sig[0] = sig_array[0]
    for i in range(1, len(sig_array)):
        filtered_sig[i] = alpha * sig_array[i] + (1 - alpha) * filtered_sig[i - 1]
    return filtered_sig

for i in range(len(L_path_in)):
    print(f"Fichier : {L_path_in[i]} : ...")
    L_temps, sig_brut = lire_fichier(L_path_in[i])
    write_file(L_path_out[i], L_temps, filtre_passe_bas_1er_ordre(sig_brut, L_temps))
    print(f"Fichier : {L_path_in[i]} : done")
```



II.II. Amplitude d'un signal périodique

```python
import numpy as np
import matplotlib.pyplot as plt

def write_file(PATH, List_temps, List_sig):
    with open(PATH, 'w') as file:
        file.write('Time;Signal\n')
        for t, s in zip(List_temps, List_sig):
            file.write(f"{t}\t{s}\n")

def get_amplitude(PATH):
    List_temps = []
    List_sig = []
    file = open(PATH)

    L_lignes = file.readlines()
    for ligne in L_lignes[1:70000]:
        ligne = ligne.strip().split('\t')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace(',', '.') for x in ligne]
        List_temps.append(float(ligne[0]))
        List_sig.append(float(ligne[1]))

    amplitude = max(List_sig) - min(List_sig)
    return amplitude

for i in range(len(L_path)):
    amp = get_amplitude(L_path[i])
    write_file('Python/Code/amplitudes.csv', L_labels[:i+1], [get_amplitude(p) for p in
L_path[:i+1]])
```

III. Données et valeurs numériques

Notations	Valeur	Unité	Description
μ_0	$4\pi.10^{-7}$	$H.m^{-1}$	Perméabilité magnétique du vide
μ_r	3,5 (ODG)	-	Perméabilité relative de l'Alnico 5
$e(t)$	-	V	Force électromotrice (f.e.m.) induite dans le micro
$u(t)$	-	V	Tension en sortie du micro chargé
$u_{GBF}(t)$	-	V	Tension délivré par le générateur de fonction
$H(p)$	-	-	Fonction de transfert du circuit de filtrage
R_{s_S}	$5{,}26.10^3$	Ω	Resistance interne du micro SingleCoil
L_S	2,43	H	Inductance du micro SingleCoil
C_S	360.10^{-12}	F	Capacité interne du micro SingleCoil
R_{p_S}	500.10^3	Ω	Resistance des pertes par courant de Foucault
$Z_S(\omega)$	-	Ω	Impédance complexe du micro SingleCoil
$R_{s_{Hp}}$	$4{,}52.10^3$	Ω	Resistance interne du micro Humbucker parallèle
L_{Hp}	1,77	H	Inductance du micro Humbucker parallèle
C_{Hp}	50.10^{-12}	F	Capacité interne du micro Humbucker parallèle
$R_{p_{Hp}}$	351.10^3	Ω	Resistance des pertes par courant de Foucault
$Z_{Hp}(\omega)$	-	Ω	Impédance complexe du micro Humbucker parallèle
$R_{s_{Hs}}$	$15{,}10.10^3$	Ω	Resistance interne du micro Humbucker série
L_{Hs}	4,71	H	Inductance du micro Humbucker série
C_{Hs}	110.10^{-12}	F	Capacité interne du micro Humbucker série
$R_{p_{Hs}}$	$1{,}07.10^6$	Ω	Resistance des pertes par courant de Foucault
$Z_{Hs}(\omega)$	-	Ω	Impédance complexe du micro Humbucker série
$z(x,t)$	-	m	Déplacement vertical de la corde
z_0	0,004	m	Distance corde micro au repos
L	0,85	m	Longueur de la corde vibrante
x_0	0,300	m	Position de l'origine de la perturbation
A_0	0,001	m	Amplitude initiale de la perturbation
B_n	-	m	Amplitude de l'harmonique de rang n
F	80	Hz	Fréquence fondamentale de vibration de la corde
m	-	$A.m^{-2}$	Moment dipolaire de la corde

Notations	Valeur	Unité	Description
$\Phi(z,t)$	-	Wb	Flux magnétique traversant la bobine
χ	100 (ODG)	-	Susceptibilité magnétique de la corde
V	1.10^{-8} (ODG)	m^3	Volume de la portion de la corde active
$B_m(z)$	-	T	Champ magnétique statique des micros
$B_c(z)$	-	T	Champ magnétique induit par le moment dipolaire
x_S	0,105	m	Position du micro SingleCoil
x_{Hp_1}, x_{Hp_2}	0,147 – 0,165	m	Position des bobinages du Humbucker parallèle
x_{Hs_1}, x_{Hs_2}	0,202 – 0,220	m	Position des bobinages du Humbucker série
Δ	0,018	m	Entraxe entre les deux bobine du Humbucker
F_B	50	Hz	Fréquence du champ magnétique parasite
B_0	-	T	Amplitude du champ magnétique parasite
α	2 (ODG)	m^{-1}	Gradient du champ magnétique parasite
k_S	1,46	$mV.\mu T^{-1}$	Sensibilité théorique du SingleCoil
k_{Hp}	0,02	$mV.\mu T^{-1}$	Sensibilité théorique du Humbucker parallèle
k_{Hs}		$mV.\mu T^{-1}$	Sensibilité théorique du Humbucker série
k_{Sexp}	1,48	$mV.\mu T^{-1}$	Sensibilité expérimentale du SingleCoil
k_{Hpexp}	0,62	$mV.\mu T^{-1}$	Sensibilité expérimentale du Humbucker parallèle
k_{Hsexp}		$mV.\mu T^{-1}$	Sensibilité expérimentale du Humbucker série
$CMRR$	37	dB	CMRR théorique du Humbucker
$CMRR_{Hpexp}$	7,6	dB	CMRR expérimentale du Humbucker parallèle
$CMRR_{Hsexp}$		dB	CMRR expérimentale du Humbucker série